BTK AKADEMİ

İLERİ SEVİYE PYTHON PROGRAMLAMA DİLİ

Düzenli İfadeler (Regular Expressions)

EBRU AYDOĞAN

Regex Felsefesi – Neden Farklı?

- **Kavram:** Regex bir programlama dili değildir, bir **"Pattern Matching" (Desen Eşleştirme)** dilidir.
- **Farkı:** string.replace() veya split() gibi yöntemler statik veriler için iyidir. Regex ise **dinamik/belirsiz** veriler (örneğin: "herhangi bir 10 haneli sayı", "herhangi bir @ ile biten e-posta") için tasarlanmıştır.

BTK AKADEMİ

re Modülü Fonksiyonları

Fonksiyon	Kullanım Yeri	İpucu
re.match()	Metnin **sadece** başından itibaren arar.	En hızlısıdır, ama sadece başa bakar.
re.search()	Metnin **herhangi bir yerinde** ilk eşleşeni bulur.	Koşullu kontrollerde (if) kullanılır.
re.findall()	Eşleşenlerin tümünü liste olarak döndürür.	Veri çekme/listeleme için şarttır.
re.finditer()	Eşleşenlerin tümünü **Iterator** (nesne) olarak döner.	Çok büyük verilerde RAM dostudur.
re.sub()	Bulunanları başka bir şeyle değiştirir.	"Temizlik" operasyonları için kullanılır.

Karakter Sınıfları (Temel Alfabe)

Kod	Açıklama	Örnek
.	Yeni satır hariç her karakter	a.b -> "acb", "a1b"
\d	Tüm rakamlar (0-9)	\d+ -> 123
\D	Rakam olmayanlar	\D -> "a", "!"
\w	Harf, rakam, alt tire (A-Z, a-z, 0-9, _)	\w -> "kod_1"
\W	Harf/Rakam olmayanlar (boşluk, nokta vb.)	\W -> " "
\s	Boşluk, tab, yeni satır	\s+ -> Boşlukları yakalar

BTK AKADEMİ

Nicelik Belirleyiciler (Quantifiers)

Kod	Açıklama	Örnek
*	0 veya daha fazla	a* -> "", "a", "aa"
+	1 veya daha fazla	a+ -> "a", "aa"
?	0 veya 1 (İsteğe bağlı)	a? -> "", "a"
{n}	Tam n tane	\d{3} -> "123"
{n,m}	n ile m arası	\d{2,4} -> "12", "123", "1234"

BTK AKADEMİ

Sınırlar ve Gruplama (Anchors & Groups)

- Karmaşık metinlerde "tam olarak nereyi" hedeflediğimizi belirtiriz.
- **Sınırlar:**
  - ^ : Satır başı. (^Merhaba -> Satırın en başında Merhaba olmalı)
  - $: Satır sonu. (bitti$ -> Satırın en sonunda bitti olmalı)
  - \b : Kelime sınırı (Boşluk veya noktalama). (\btest\b -> "test" kelimesini bulur, "testing" içinde bulmaz!)
- **Gruplama () :** Eşleşmenin belirli bir parçasını yakalamak için kullanılır. ((\d{3})-(\d{3}) -> 0532-123 ifadesinde iki grubu ayrı ayrı döndürür).

BTK AKADEMİ

Örnek 1 – Veri Doğrulama (Validation)

- Bir metin içinden telefon numarasını veya e-postayı çıkartalım.

```
import re

text = "İletişim: 0532-123-4567 veya destek@btk.gov.tr"

# Telefon: 4 rakam - 3 rakam - 4 rakam
phone_pattern = r"\d{4}-\d{3}-\d{4}"
# Email: Kelime @ kelime . kelime
email_pattern = r"\w+@\w+\.\w+"

print(re.findall(phone_pattern, text))
print(re.findall(email_pattern, text))
```

BTK AKADEMİ

Örnek 2 – Veri Temizleme (Data Cleaning)

- Dosyadan okuduğumuz veride gereksiz karakterler (semboller) varsa onları temizleyelim.

```
import re

raw_data = "Fiyat: $1,200.50 | İndirim: %10"

# Rakamlar, nokta ve virgül dışındaki her şeyi sil (replace)
# [^...] ifadesi 'içindekiler hariç' demektir.
clean_data = re.sub(r"[^0-9.,]", "", raw_data)
print(clean_data) # "1,200.50"
```

BTK AKADEMİ